D H A R

Volume III — Ecosystem, Trust & Adoption

TEMPERING DHAR

"Trust that survives contact with an adversary, not just a demo."

Document	Tempering Dhar — Security, Assurance & Defence-Grade Engineering
Version	1.0
Date	September 8, 2026
Status	Ratified
Part of	The Canon of Dhar, Volume III
Depends on	Spec1 §12–15, Stage-delta §3–4, Stage-epsilon
Audience	Security reviewers, government & defence evaluators

1. Mandate

State, precisely and without inflation, what Dhar's design guarantees for security and safety, what it does not, and what would be required to demonstrate either claim to an outside evaluator. Spec0 §5 names defence and critical national infrastructure as target domains; this document is where that claim is made accountable rather than aspirational. Nothing here asserts a certification Dhar does not yet hold — software that does not exist yet cannot be certified, and claiming otherwise would undermine exactly the credibility this document exists to build.

2. Threat Model

2.1 Assets & adversaries

Asset	Representative adversary	What they want
The compiler binary itself	Supply-chain attacker; nation-state actor	Insert a backdoor that survives into every program the compiler builds
Kun's registry & packages	Typosquatter; compromised maintainer account	Distribute malicious code as a trusted dependency
A compiled Dhar program's memory	Remote attacker with network access	Corrupt memory to achieve code execution or data exfiltration
Sensitive runtime data (keys, credentials)	Local attacker; forensic recovery after disposal	Recover secrets from memory, swap, or crash dumps
Cryptographic operations	Adversary with timing/cache measurement access	Recover key material via side channel

2.2 Attack surface

Four layers, each with a different owner in this Canon:

1. **Compiler-introduced.** The classic case is Ken Thompson's 1984 Turing Award lecture, “Reflections on Trusting Trust”: a compiler can be made to insert a backdoor into everything it compiles, including future versions of itself, leaving no trace in the source it was built from. A from-scratch, self-hosting compiler (Stage-alpha through Stage-beta) is exactly the kind of software this attack targets. Addressed in §5.3.

2. **Supply-chain.** A compromised dependency, registry, or build environment. Owned by Kun (Stage-delta §3.7) and Stage-epsilon's reproducible builds and signing; recapped in §5.

3. **Runtime / memory-safety.** Vulnerabilities in compiled Dhar programs themselves — the class of bug Spec0 §2.2's Java-class-robustness commitment targets directly. Addressed in §3.

4. **Side-channel.** Timing, cache, and power-analysis leakage from cryptographic or otherwise sensitive code, which memory safety alone does not address. Addressed in §6.

3. Memory & Type Safety: What's Eliminated By Construction

In November 2022 the NSA recommended memory-safe languages; in December 2023, CISA and the NSA jointly published *The Case for Memory Safe Roadmaps*, calling on software manufacturers serving critical infrastructure to publish a memory-safety roadmap; in June 2025 the two agencies followed with practical adoption guidance. Google and Microsoft have both reported that most vulnerabilities in their own products trace back to memory-safety bugs, and Apple's own review of its operating systems found the same pattern. This is not a theoretical concern this Canon is retrofitting a justification for; it is the specific, named problem Dhar's memory model (Spec1 §14) was designed against from the start.

The frequently cited gap in that guidance is real: established memory-safe languages (Go, Java, C#, Python) buy their safety with a garbage collector and a runtime, which is exactly what Spec0 §2.2 rules out for Dhar. Rust and Ada/SPARK are the languages usually named as filling that gap; Dhar's thesis (Spec0 §3) is to occupy the same ground, independently.

Vulnerability class	CWE	Eliminated by
Buffer overflow (classic, stack, heap)	CWE-120 / 121 / 122	Mandatory bounds checking on every array and raw[N] access (Spec1 §14.4) — an out-of-range index is a diagnosable failure, never silent adjacent-memory access
NULL pointer dereference	CWE-476	own T's absence is a checked None variant (Spec1 §14.3), not an unchecked null; the check is enforced at match time, not left to the programmer to remember
Use-after-free	CWE-416	Compiler-inserted deallocation under single ownership (Spec1 §14.1) — there is no manual free for a use-after-free to follow
Double free	CWE-415	Same mechanism: deallocation is inserted exactly once, by the compiler, tied to the single owner going out of scope
Unchecked return value / ignored error	CWE-252	state must be explicitly trapped (Spec1 §16.1–16.2); an ignored Error case is a compile-time gap, not a silent runtime one
Missing case / incomplete handling	(no single CWE; a root cause behind many)	alloy + match must be exhaustive (Spec1 §11) — a forgotten variant is a compile error

This table is a claim about the language's design, not a claim about any particular program written in it, and not a claim that these CWE categories are the only ones that matter — injection, access-control, and authentication weaknesses (also CWE Top-25 regulars) are application-logic concerns no language design eliminates by itself.

3.1 What remains the programmer's, or the tooling's, responsibility

Every capability that can violate the guarantees above is a named, individually greppable keyword rather than one blanket escape hatch: `peek`/`poke` (raw memory), `asm` (inline assembly), and `foreign` (calling code outside Dhar's guarantees entirely). A security review of a Dhar codebase can locate every place the language's guarantees are suspended with four search terms, which is a sharper audit surface than a single `unsafe` block covering several unrelated kinds of risk at once (Spec1 §14, §18–19). Logic errors, algorithmic complexity, and injection/access-control bugs are unaffected by any of this — memory safety is necessary for a defence-grade toolchain, not sufficient.

4. Concurrency Safety

Spec1 §15 requires that `view`/`grab` exclusivity (Spec1 §14.2) hold across `spark`/`draw` task boundaries, not only within sequential code. Practically: a value cannot be mutated from two concurrently running tasks at once, by the same construction that already prevents two overlapping mutable references in ordinary sequential code. Data races (CWE-362, race condition) are closed as a special case of aliasing, not bolted on as a separate concurrency-checking pass — the same design economy Spec1 §10.5 already applies to logical and symbolic programming.

5. Supply-Chain Security & Provenance

5.1 What Kun already guarantees

Recapping Stage-delta §3.7 rather than restating it: every published package carries a content hash and signature verified on install; resolution is deterministic and lockfiles prevent silent re-resolution; a private, self-hosted registry is first-class, not an afterthought, specifically so a critical-infrastructure adopter's supply chain does not depend on a single externally controlled service.

5.2 Reproducible builds as continuous audit

Stage-epsilon requires two independent builds of the same source, on different machines, to be bit-for-bit identical. This is not only a quality property: it means any third party can independently verify that a released binary actually corresponds to its published source, rather than trusting the publisher's word — the software equivalent of the Common Criteria requirement that evaluators reproduce the evidence themselves rather than accept a vendor's claims.

5.3 The Trusting Trust problem

Stage-beta's self-hosting proof (byte-for-byte identical output across two generations, Stage-beta §4) establishes that Stage-beta reproduces itself correctly; it does not, by itself, prove the compiler is free of a Thompson-style backdoor inserted at the Stage-alpha bootstrap and silently propagated ever since, since a corrupted compiler can reproduce a corrupted copy of itself perfectly. The established academic mitigation is Diverse Double-Compiling (David A. Wheeler, 2005 PhD thesis): compiling Stage-beta's source with a second, independently written compiler and confirming the two resulting binaries are equivalent. This Canon does not yet mandate Diverse Double-Compiling as an acceptance criterion — doing so honestly requires a second independent compiler to exist — but records it here as the specific, named hardening step a future revision of Stage-epsilon should require before any claim of Trusting-Trust resistance is made.

6. Sensitive Data Handling

6.1 Constant-time operations

std.crypto (Stage-delta §2) **must** implement comparison and branching over secret data (key material, MACs, password hashes) in constant time with respect to the secret's value, closing timing side channels of the kind that have repeatedly broken otherwise mathematically sound cryptographic implementations. This is a requirement on std.crypto's implementation discipline, not a new language keyword.

6.2 Zero-on-drop

A mold holding sensitive data (keys, plaintext passwords, unencrypted PII) should be able to declare that its memory is overwritten, not merely deallocated, when its owning binding goes out of scope — closing the gap where freed-but-uncleared memory is later reused and its previous contents recovered. This is identified here as a required std.crypto/std.mem capability for Stage-delta rather than specified in grammar-level detail in this document, consistent with the Master Index §3.6 rule that a keyword's semantics are ratified in exactly one place.

7. Certification Readiness

None of the following are claims that Dhar currently holds any certification — it cannot, since Stage-alpha through Stage-epsilon are not yet built. This section states which frameworks are relevant, what each actually requires, and which of Dhar's already -ratified design choices position it well versus which stages have not yet produced the evidence a real evaluation would need.

Framework	What it actually is	Where Dhar stands
Common Criteria (ISO/IEC 15408)	International standard for security evaluation, seven Evaluation Assurance Levels (EAL1–EAL7); requires an accredited lab to evaluate a specific, finished product against a defined Protection Profile.	Not applicable until a finished, versioned toolchain exists to submit; Spec1's exhaustiveness and bounds-checking guarantees (§3) are the kind of property an EAL evaluation would examine first.
NIST SSDF (SP 800-218)	Outcome-based secure-development practices in four families (Prepare the Organization, Protect the Software, Produce Well-Secured Software, Respond to Vulnerabilities); conformance is by vendor attestation, not third-party certification; required to sell software to the US federal government under Executive Order 14028.	The Guild of Dhar (governance) and this document's §8 (vulnerability disclosure) are direct SSDF-family inputs once there is an organization to attest on behalf of.
FIPS 140-3	Validates specific cryptographic modules (not whole languages or toolchains) against approved algorithms (AES, SHA-2, RSA, ECDSA and similar) through NIST and the Canadian Centre for Cyber Security's joint CMVP program.	Relevant only to std.crypto's eventual implementation (Stage-delta), evaluated as its own module, not to Dhar the language.
DO-178C / IEC 61508 / ISO 26262	Sector-specific software assurance for airborne systems, industrial functional safety, and automotive functional safety respectively; each requires rigorous, documented traceability from requirement to code to test.	Aspirational for the embedded/hardware use case (Spec0 §5); would require Stage-epsilon's static analysis and Stage-delta's testing infrastructure to produce the traceability evidence these standards demand.

The honest summary: Dhar's language-level design choices (§3) are the kind of evidence these frameworks look for, and are being made with that evaluation in mind, but none of them can be pursued in earnest before Stage-epsilon exists to produce, and Stage-delta to test, the artifact actually being evaluated.

8. Vulnerability Disclosure & Incident Response

- A published, monitored channel for reporting suspected vulnerabilities, separate from ordinary bug reports, following coordinated disclosure norms (private report, agreed remediation window, public advisory on fix).
- Every fixed vulnerability affecting a released version gets a CVE and a public security advisory, cross-referencing the affected Kun package version(s) directly — a defence or critical-infrastructure adopter needs to know which locked dependency versions are affected without re-deriving it themselves.
- A security advisory states affected versions, the fixed version, and a workaround if one exists, in that order, before any narrative explanation.

9. Acceptance Criteria

1. Every claim in §3's table is verified by at least one test in Stage-beta's suite (per Stage-beta §4) that deliberately attempts the eliminated bug class and confirms it is caught, not merely a test that the happy path works.

2. The concurrency guarantee in §4 has a test that deliberately attempts to alias a grab reference across a spark boundary and confirms it is rejected at compile time.

3. A published vulnerability-disclosure channel and process (§8) exists before Stage-epsilon's first tagged release, not after.

4. Diverse Double-Compiling (§5.3) is re-evaluated as a mandatory Stage-epsilon acceptance criterion once a second independent Dhar implementation exists, and this document is amended accordingly rather than left silently outdated.

10. Relationship to Other Volumes

This document defines	Ratified in	Enforced/tested in
Memory & type safety guarantees	Spec1 §12–14	Stage-beta §4 acceptance tests
Concurrency safety	Spec1 §15	Stage-beta §4 / Stage-gamma
Supply-chain guarantees	Stage-delta §3.7	Stage-epsilon reproducible builds & signing
Certification evidence	(this document, §7)	Stage-epsilon static analysis & Stage-delta testing

---

Tempering Dhar — Security, Assurance & Defence-Grade Engineering · v1.0 · September 8, 2026 · Ratified. Grounded in NSA/CISA memory-safety guidance (2022–2025), NIST SP 800-218 (SSDF v1.1, Feb 2022), FIPS 140-3 (effective 2019), and Common Criteria (ISO/IEC 15408). Revision log: v1.0 — first ratified edition; companion amendment to Spec1 §15 (view/grab exclusivity across spark/draw) made concurrently.